

Relational model: Overview

1

Relational Model

- Relational Data Model was first proposed by **Ted Codd** of IBM in the **1970s**. Its commercial implementations were observed in the **1980s**.
- The relational data model is employed for **storing and processing** the data in the database.
- **Relational Model** represents the database as a collection of relations.
- A relation is nothing but a table of values.

Basic terminologies used in relational data model

- **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME, etc.
- **Tables** – In the Relational model the, relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.
- **Tuple** – It is nothing but a single row of a table, which contains a single record.
- **Relation Schema:** A relation schema represents the name of the relation with its attributes.
- **Degree:** The total number of attributes which in the relation is called the **degree** of the relation.
- **Cardinality:** Total number of rows present in the Table.
- **Column:** The column represents the set of values for a specific attribute.
- **Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
- **Relation key** – Every row has one, two or multiple attributes, which is called relation key, which uniquely identifies each tuple in the relation.
- **Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain.
- **Null** – The value which is unknown or unavailable is called NULL value. It is represented as blank space.
- Figure shows an example of a **STUDENT** relation, which corresponds to the STUDENT schema just specified. Each tuple in the relation represents a particular student entity

Relation Name		Attributes						
STUDENT								
		Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Tuples		Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21
		Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
		Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
		Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
		Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25

The attributes and tuples of a relation STUDENT.

characteristics

1. Each **relation** in a database must have a **distinct or unique name** which would separate it from the other relations in a database.
2. A relation must **not** have **two attributes** with the **same name**. Each attribute must have a distinct name.

3. Duplicate tuples must **not** be present in a relation.

Roll_No.	Name	Department
101	Steive	Comp. Sci.
265	Jhoson	Finance
505	Margret	Biology
505	Margret	Biology
256	Davis	Comp. Sci.
453	Sheryl	Biology
365	Emma	Maths



Student Relation in Relational Model

4. Each tuple must have exactly **one data value** for an **attribute**. For example, below in the first table, you can see that for Roll_No. 265 we have enrolled two students Jhoson and Charles, this would not work. We must have only one student for one Roll_No.

Roll_No.	Name	Department	Roll_No.	Name	Department
101	Steive	Comp. Sci.	101	Steive	Comp. Sci.
265	Jhoson	Finance	265	Jhoson	Finance
	Charles	Finance	346	Charles	Finance
505	Margret	Biology	505	Margret	Biology
325	Jenny	Social Sci.	325	Jenny	Social Sci.
256	Davis	Comp. Sci.	256	Davis	Comp. Sci.
453	Sheryl	Biology	453	Sheryl	Biology
365	Emma	Maths	365	Emma	Maths



5. Tuples in a relation do not have to follow a significant order as the relation is not order-sensitive.

6. The attributes of a relation also do not have to follow certain ordering, it's up to the developer to decide the ordering of attributes.

Constraints: types

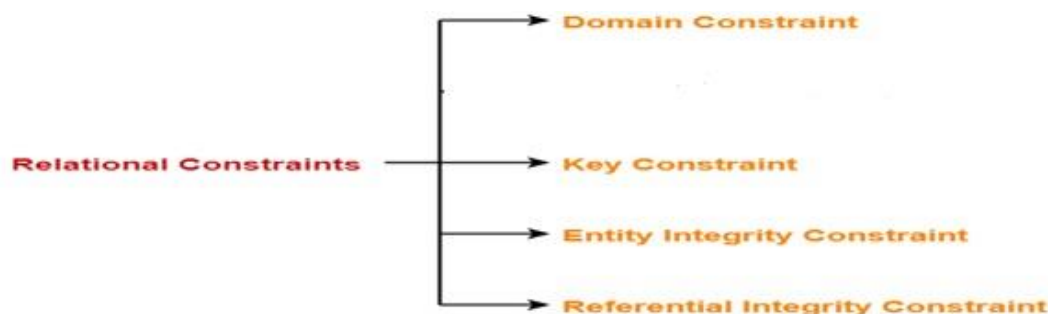
Constraints in DBMS-

Relational constraints are the restrictions or conditions imposed on the database contents and operations.

They ensure the correctness of data in the database.

Types of Constraints in DBMS-

In DBMS, there are following 5 different types of relational constraints-



1. Domain constraint
2. Key constraint
3. Entity Integrity constraint
4. Referential Integrity constraint

Domain Constraint-

Domain constraint defines the domain or set of values for an attribute.

1. Every domain must contain atomic values (smallest indivisible units) it means composite and multi-valued attributes are not allowed.
2. We perform datatype check here, which means when we assign a data type to a column, we limit the values that it can contain. Eg. If we assign the datatype of attribute age as int, we cant give it values other then int datatype.

Example-

Consider the following Student table-

STU_ID	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
S004	Rahul	A

Here, value 'A' is not allowed since only integer values can be taken by the age attribute.

Key Constraints or Uniqueness Constraints :

1. These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
2. A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.
3. Null values are not allowed in the primary key, hence Not Null constraint is also a part of key constraint.

Example-01:

Consider the following Student table-

STU_ID	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
S004	Rahul	20

This relation satisfies the tuple uniqueness constraint since here all the tuples are unique.

Example-02:

Consider the following Student table-

STU_ID	Name	Age
S001	Akshay	20
S001	Akshay	20
S003	Shashank	20
S004	Rahul	20

This relation does not satisfy the tuple uniqueness constraint since here all the tuples are not unique.

Entity Integrity Constraint-

Entity integrity constraint specifies that no attribute of primary key must contain a null value in any relation.

This is because using primary key we identify each tuple uniquely in a relation.

Example-

Consider the following Student table-

STU_ID	Name	Age
S001	Akshay	20
S002	Abhishek	21
S003	Shashank	20
	Rahul	20

This relation does not satisfy the entity integrity constraint as here the primary key contains a NULL value.

Referential Integrity Constraint-

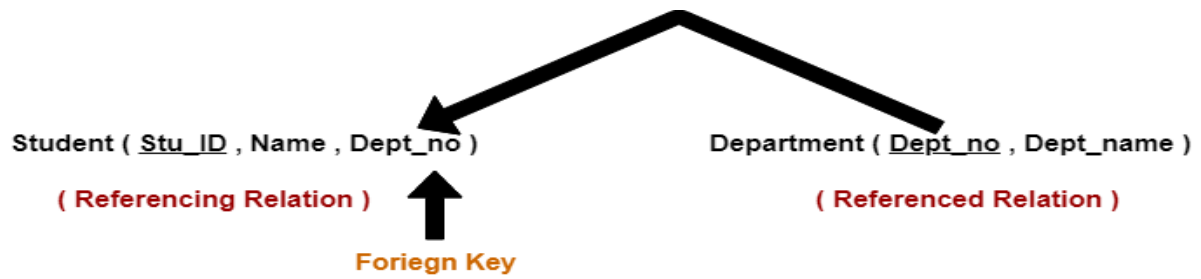
The Referential integrity constraints is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.

1. This constraint is enforced through foreign key, when an attribute in the foreign key of relation R1 have the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.
2. The values of the foreign key in a tuple of relation R1 can either take the values of the primary key for some tuple in relation R2, or can take NULL values, but can't be empty.

Example-

Consider the following two relations- 'Student' and 'Department'.

Here, relation 'Student' references the relation 'Department'.



Student

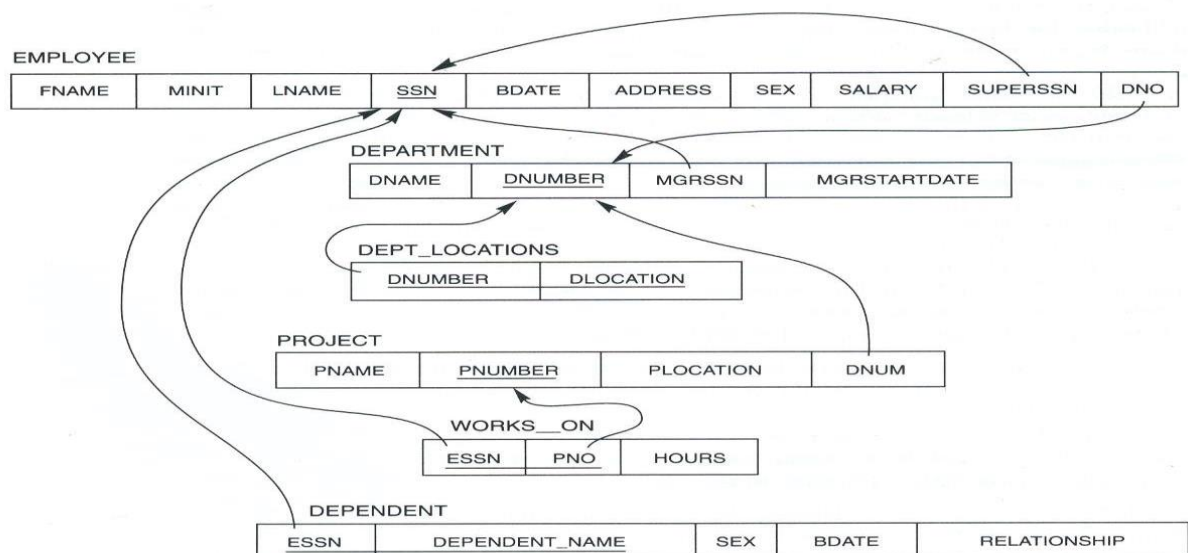
STU_ID	Name	Dept_no
S001	Akshay	D10
S002	Abhishek	D10
S003	Shashank	D11
S004	Rahul	D14

Department

Dept_no	Dept_name
D10	ASET
D11	ALS
D12	ASFL
D13	ASHS

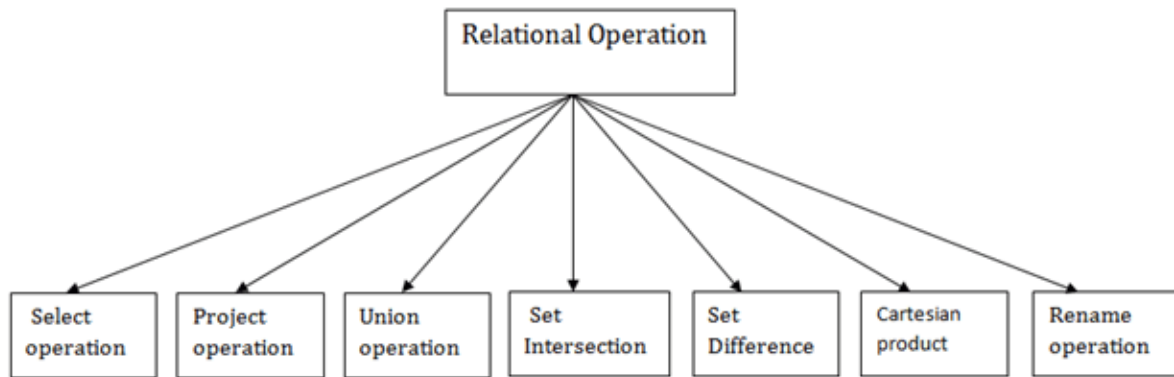
Here,

- The relation 'Student' does not satisfy the referential integrity constraint.
- This is because in relation 'Department', no value of primary key specifies department no. 14.
- Thus, referential integrity constraint is violated.



Referential integrity constraints displayed on the COMPANY relational database schema.

Types of Relational operation



1. Select Operation:

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma (σ).
- 1. Notation: $\sigma p(r)$

Where:

σ is used for selection prediction

r is used for relation

p is used as a propositional logic formula which may use connectors like: AND OR and NOT.

These relational can use as relational operators like $=$, $\neq$, $\geq$, $<$, $>$, $\leq$.

For example: LOAN Relation

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input:

1. σ BRANCH_NAME="perryride" (LOAN)

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

2. Project Operation:

- This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.
 - It is denoted by Π .
1. Notation: $\Pi A_1, A_2, A_n (r)$

Where

A1, A2, A3 is used as an attribute name of relation **r**.

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison
Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye
Johnson	Alma	Brooklyn
Brooks	Senator	Brooklyn

Input:

1. $\Pi \text{NAME, CITY (CUSTOMER)}$

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye
Johnson	Brooklyn
Brooks	Brooklyn

3. Union Operation:

- Suppose there are two relations R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
 - It eliminates the duplicate tuples. It is denoted by $\cup$.
1. Notation: $R \cup S$

A union operation must hold the following condition:

- R and S must have the attribute of the same number.
- Duplicate tuples are eliminated automatically.

Example:

DEPOSITOR RELATION

CUSTOMER_NAME	ACCOUNT_NO
---------------	------------

Johnson	A-101
Smith	A-121
Mayes	A-321
Turner	A-176
Johnson	A-273
Jones	A-472
Lindsay	A-284

BORROW RELATION

CUSTOMER_NAME	LOAN_NO
Jones	L-17
Smith	L-23
Hayes	L-15
Jackson	L-14
Curry	L-93
Smith	L-11
Williams	L-17

Input:

1. $\prod \text{CUSTOMER_NAME (BORROW)} \cup \prod \text{CUSTOMER_NAME (DEPOSITOR)}$

Output:

CUSTOMER_NAME
Johnson
Smith
Hayes
Turner
Jones
Lindsay
Jackson
Curry
Williams

4 Set Intersection:

- Suppose there are two relations R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection $\cap$.
- 1. Notation: $R \cap S$

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. Π CUSTOMER_NAME (BORROW) $\cap$ Π CUSTOMER_NAME (DEPOSITOR)

Output:

CUSTOMER_NAME
Smith
Jones

5. Set Difference:

- Suppose there are two relations R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus (-).
- 1. Notation: $R - S$

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. Π CUSTOMER_NAME (BORROW) - Π CUSTOMER_NAME (DEPOSITOR)

Output:

CUSTOMER_NAME
Jackson
Hayes
Willians
Curry

6. Cartesian product

- The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.
- It is denoted by X.
- 1. Notation: $E \times D$

Example:

EMPLOYEE

EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

DEPARTMENT

DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Input:

1. EMPLOYEE X DEPARTMENT

Output:

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

7. Rename Operation:

The rename operation is used to rename the output relation. It is denoted by **ρ** (ρ).

Example: We can use the rename operator to rename STUDENT relation to STUDENT1.

1. $\rho(\text{STUDENT1}, \text{STUDENT})$

Advantages and Disadvantages

Advantages

- **Simplicity:** A Relational data model in DBMS is simpler than the hierarchical and network model.
- **Structural Independence:** The relational database is only concerned with data and not with a structure. This can improve the performance of the model.
- **Easy to use:** The Relational model in DBMS is easy as tables consisting of rows and columns are quite natural and simple to understand
- **Query capability:** It makes possible for a high-level query language like [SQL](#) to avoid complex database navigation.
- **Data independence:** The Structure of Relational database can be changed without having to change any application.
- **Scalable:** Regarding a number of records, or rows, and the number of fields, a database should be enlarged to enhance its usability.

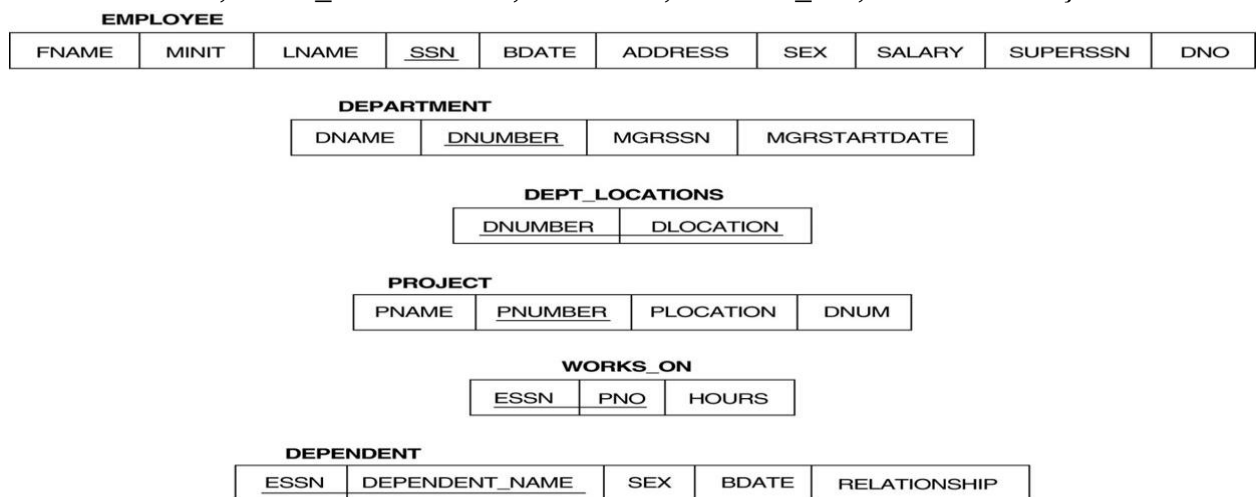
Disadvantages

- Few relational databases have limits on field lengths which can't be exceeded.
- Relational databases can sometimes become complex as the amount of data grows, and the relations between pieces of data become more complicated.
- Complex relational database systems may lead to isolated databases where the information cannot be shared from one system to another.

Relational Databases and Relational Database Schemas

Relational Databases and Relational Database Schemas

- A relational database usually contains many relations, with tuples in relations that are related in various ways.
- A **relational database schema** S is a set of relation schemas $S = \{R_1, R_2, \dots, R_m\}$ and a set of **integrity constraints** IC . A **relational database state** DB of S is a set of relation states $DB = \{r_1, r_2, \dots, r_m\}$ such that each r_i is a state of R_i and such that the r_i relation states satisfy the integrity constraints specified in IC .
- Figure 3.3 shows a relational database schema that we call $COMPANY = \{EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT\}$.



Schema diagram for the COMPANY relational database schema.

Design anomalies

An anomaly is an irregularity, or something which deviates from the expected or normal state. When designing databases, we identify three types of anomalies: Insert, Update and Delete. The operations of the relational model can be categorized into retrievals and updates. There are three basic operations database **modification** or **update** operations.

- **Insert,**
- **Delete, and**
- **Update (or Modify).**

Insert is used to insert one or more new tuples in a relation,

Delete is used to delete tuples,

Update (or Modify) is used to change the values of some attributes in existing tuples.

Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated.

INSERT Anomaly in Database

Student_ID	Enroll Number	Roll Number	Name	Address	City	Dept_ID
1	AX001	1-BSCS-2018	John	Street 13 House 14	Lahore	3
2	AX002	2-BSBT-2018	Faiz	house 18 Defence Club	Karachi	4
3	AX003	3-BSEN-2018	Nouman	Street 19 House 20	Faislabad	5
4	AX004	4-PEEN-2018	Jerry	Street 18 House 29	Dubai	6

Dept_Id	Name	Phone Extension
3	Computer Science	398974
4	Botany	988784
5	English	898418

An Insert Anomaly occurs when attributes cannot be inserted into the database without the presence of other attributes. Usually when a child is inserted without parent.

Jerry is a new Student with department id 6. There is no Department with this Dept_ID 6. Hence , the anomaly. The usual behaviour should be a new department id with 6 and only then Student could have it.

UPDATE Anomaly in Database

Student_ID	Enroll Number	Roll Number	Name	Address	City	Dept_ID
1	AX001	1-BSCS-2018	John	Street 13 House 14	Lahore	3
2	AX002	2-BSBT-2018	Faiz	house 18 Defence Club	Karachi	4
3	AX003	3-BSEN-2018	Nouman	Street 19 House 20	Faislabad	5
4	AX004	4-PEEN-2018	Jerry	Street 18 House 29	Dubai	6

Dept_Id	Name	Phone Extension
3	Computer Science	398974
4	Botany	988784
8	English	898418

When duplicated data is updated at one instance and not across all instances where it was duplicated. That's an update anomaly . See below English department has now Dept_ID 8 , but unfortunately it was not updated in Student table.

DELETE Anomaly in Database

Student_ID	Enroll Number	Roll Number	Name	Address	City	Dept_ID
2	AX002	2-BSBT-2018	Faiz	house 18 Defence Club	Karachi	4
3	AX003	3-BSEN-2018	Nouman	Street 19 House 20	Faislabad	5
4	AX004	4-PEEN-2018	Jerry	Street 18 House 29	Dubai	6

Dept_Id	Name	Phone Extension
4	Botany	988784
8	English	898418

Now if someone decides to delete Computer Science department , he may end up deleting all student's data who had the department of Computer Science. So to say deletion of some attribute which causes deletion of other attributes is deletion anomaly.

These anomalies are addressed by Normalization . The normalization makes sure that all these three issues and other possible be addressed at the time of designing.

The Transaction Concept

A transaction is an executing program that includes some database operations, such as reading from the database, or applying insertions, deletions, or updates to the database.

At the end of the transaction, it must leave the database in a valid or consistent state that satisfies all the constraints specified on the database schema.

Features of good DB design

1. We should be able to **store all kinds of data** that exist in this real world. Since we need to work with all kinds of data and requirements, the database should be strong enough to store all kinds of data that are present around us.
2. We should be **able to relate the entities/tables in the database by means of relation**. i.e.; any two tables should be related. Let us say, an employee works for a department. This implies

	that an Employee is related to a particular department. We should be able to define such a relationship between any two entities in the database. There should not be any table lying without any mapping. 3. Data and applications should be isolated. Because the database is a system that gives the platform to store the data, and the data is the one that allows the database to work. Hence there should be a clear differentiation between them. 4. There should not be any duplication of data in the database. Data should be stored in such a way that it should not be repeated in multiple tables. If repeated, it would be an unnecessary waste of DB space, and maintaining such data becomes chaos. 5. DBMS has a strong query language. Once the database is designed, this helps the user to retrieve and manipulate the data. If a particular user wants to see any specific data, he can apply as many filtering conditions that he wants and pull the data that he needs. 6. Multiple users should be able to access the same database, without affecting the other user. i.e.; if teachers want to update a student's marks in the Results table at the same time, then they should be allowed to update the marks for their subjects, without modifying other subject marks. A good database should support this feature. 7. It supports multiple views to the user, depending on his role. In a school database, Students will be able to see only their reports and their access would be read-only. At the same time, teachers will have access to all the students with modification rights. But the database is the same. Hence a single database provides different views to different users. 8. The database should also provide security, i.e.; when there are multiple users accessing the database, each user will have their own levels of rights to see the database. For example, an instructor who is teaching Physics will have access to see and update marks of his subject. He will not have access to other subjects. But the HOD will have full access to all the subjects. 9. The database should also support the ACID property. i.e.; while performing any transactions like insert, update and delete, the database makes sure that the real purpose of the data is not lost. For example, if a student's address is updated, then it should make sure that there is no duplicate data created nor there is any data mismatch for that student.